

TOPIC 01: NODE.JS WITH MYSQL & ACCESS METHODS

Node.js + MySQL 3-Tier Architecture (Flowchart):

Client, Server, aur MySQL Database ke beech communication ka pipeline:



MySQL Database Access Karne Ke 4 Main Methods:

- **1. MySQL Workbench (GUI):** Desktop visual tool jahan buttons aur graphical UI se tables aur queries run ki jaati hain.
- **2. MySQL Command Line Interface (CLI):** Terminal me direct `mysql -u root -p` login karke fast commands chalana.
- **3. MySQL2 Package In Node.js:** Express backend code me SQL queries ko programmatically execute karna.
- **4. External SQL Script File (schema.sql):** Terminal me source `filename.sql` chala kar tables create karna.



Concept: Node.js directly database se baat nahi kar sakta; use ek **Database Driver** (jaise mysql2) ki zaroorat hoti hai jo JavaScript calls ko raw SQL protocol me convert karta hai!

TOPIC 02: FAKER PACKAGE (MOCK DATA GENERATION)

Faker Package Kya Hai Aur Kyu Chahiye?

Testing aur development me jab hume 100 ya 1000 users ka realistic data chahiye hota hai, toh manually likhna impossible hota hai. **Faker package** realistic dummy data generate karta hai (userId, username, email, password).

Installation: `npm install @faker-js/faker`

Generating Fake Users Code (VS Code Style):

```
faker-generate.js

const { faker } = require('@faker-js/faker');

// Function to generate a single random user tuple
const getRandomUser = () => {
  return [
    faker.string.uuid(),           // Unique ID
    faker.internet.username(),     // Random Username
    faker.internet.email(),        // Valid-looking Email
    faker.internet.password()      // Encrypted-style password
  ];
};

// Generate 100 fake users for bulk insertion
let usersData = [];
for (let i = 0; i < 100; i++) {
  usersData.push(getRandomUser());
}
```



Bulk Insert Secret: usersData array of arrays (tuples) banata hai, jise MySQL driver single INSERT INTO user VALUES ? query me 1 second me save kar leta hai!

TOPIC 03: MYSQL2 PACKAGE & CONNECTION SETUP

mysql2 Package Installation & Setup:

Node.js me MySQL connect karne ke liye sabse fast aur modern driver mysql2 hai:

- Terminal Command: `npm install mysql2`

```
database-connection.js

const mysql = require('mysql2');

// 1. Create Connection object with credentials
const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  database: 'test_db',
  password: 'mysql@123'
});

// 2. Connect to Database
connection.connect((err) => {
  if (err) throw err;
  console.log("Connected to MySQL successfully!");
});

// 3. Ending Connection (Graceful closing)
connection.end();
```

Connection Lifecycle: `end()` vs `destroy()`

- `connection.end()`: Graceful shutdown—pending queries ko complete hone deta hai, fir connection close karta hai.
- `connection.destroy()`: Forceful shutdown—turant connection kill kar deta hai.



Production Tip (`createPool`): Single connection ke bajaye production me `mysql.createPool()` use hota hai taaki thousands of concurrent users parallel query chala sakein!

TOPIC 04: RUNNING SQL QUERIES (connection.query)

connection.query() Method Syntax & Execution:

Node.js se database par koi bhi SQL query chalane ke liye connection.query() use hota hai:

```
run-queries.js

const q = "SHOW TABLES";

connection.query(q, (err, result, fields) => {
  if (err) {
    console.log("SQL Query Error:", err);
    return;
  }
  console.log("Query Result:", result);
});
```

Result Format (RowDataPacket Array):

- **result:** JavaScript Objects ka ek array hota hai (RowDataPacket [{ id: '1', username: 'shivam' }, ...]). Ise hum standard JS array ki tarah loop/filter kar sakte hain.
- **fields:** Columns ka metadata (table name, data types, length) store karta hai.

Async / Await Modern Alternative (mysql2/promise):

```
promise-query.js

const mysql = require('mysql2/promise');

async function fetchUsers() {
  const connection = await mysql.createConnection({ /* config */ });
  const [rows, fields] = await connection.query("SELECT * FROM user");
  console.log(rows);
}
```

TOPIC 05: PLACEHOLDERS (?) & SQL INJECTION DEFENSE

SQL Injection Ka Khatra:



Agar user input ko direct string concatenation ("SELECT * FROM user WHERE name = '" + name + "'") se jodoge, toh hacker ' OR '1'='1 daal kar poora database dump ya delete kar dega!

The Solution: Placeholders (?) & Prepared Statements

MySQL2 driver me ? placeholder use karne se driver user input ko strictly **data** ki tarah treat karta hai, code ki tarah nahi!

Single vs Bulk Insert Code:

```
insert-placeholders.js

-- 1. SINGLE INSERT (Using individual ? placeholders)
const q1 = "INSERT INTO user (id, username, email, password) VALUES (?, ?, ?, ?)";
const userData = ['101', 'shivam', 'shivam@test.com', 'pass123'];

connection.query(q1, userData, (err, res) => {
  if (err) throw err;
  console.log("1 User inserted!");
});

-- 2. BULK INSERT (Using single ? for Array of Arrays)
const q2 = "INSERT INTO user (id, username, email, password) VALUES ?";
connection.query(q2, [userData], (err, res) => {
  if (err) throw err;
  console.log(res.affectedRows + " users inserted in 1 query!");
});
```

TOPIC 06: RESTFUL ROUTING WITH MYSQL (5 CORE ROUTES)

User Resource Ke 5 Core RESTful Routes:

HTTP Method	Route Path	Purpose & Kya Karta Hai	SQL Query Used
GET	/	Home: Database me total users count dikhana	SELECT COUNT(*) FROM user
GET	/user	Index: Saare users list render karna (id, username, email)	SELECT id, username, email FROM user
POST	/user	Create: Naya user database me insert karna	INSERT INTO user VALUES (?, ?, ?, ?)
PATCH	/user/:id	Update: Specific user ka username modify karna	UPDATE user SET username = ? WHERE id = ?
DELETE	/user/:id	Destroy: Specific user ko database se delete karna	DELETE FROM user WHERE id = ?

Form Routes (Supporting HTML Views):

- GET /user/new: Naya user banane ka HTML form render karta hai.
- GET /user/:id/edit: Specific user ko edit karne ka form render karta hai (current username pre-filled).

TOPIC 07: EXECUTING SQL FILES VIA CLI (SCHEMA.SQL)

schema.sql File Workflow:

Professional projects me table schema ko code me hardcode karne ke bajaye ek `schema.sql` file banate hain:

```
CREATE TABLE IF NOT EXISTS user (  
  id VARCHAR(50) PRIMARY KEY,  
  username VARCHAR(50) UNIQUE NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  password VARCHAR(50) NOT NULL  
);
```

CLI Se schema.sql Run Karne Ke Steps:

- **Step 1:** Terminal / Command Prompt me MySQL login karo:
`mysql -u root -p` (Enter password)
- **Step 2:** Database select karo:
`USE test_db;`
- **Step 3:** File execute karo:
`source schema.sql`; ya `source /path/to/schema.sql`;



One-Line Terminal Shortcut: Bina MySQL prompt me ghuse direct terminal se import karne ke liye:

```
mysql -u root -p database_name < schema.sql
```

TOPIC 08: COMPLETE EXPRESS + MYSQL REST SERVER

Full Production-Ready index.js (VS Code Style):

```
index.js - Express + MySQL RESTful Architecture

const express = require('express');
const app = express();
const mysql = require('mysql2');
const methodOverride = require('method-override');

app.use(express.urlencoded({ extended: true }));
app.use(methodOverride('_method'));
app.set('view engine', 'ejs');

const connection = mysql.createConnection({
  host: 'localhost', user: 'root', database: 'test_db', password: 'mysql@123'
});

// 1. GET / → Show count of users
app.get('/', (req, res) => {
  connection.query('SELECT COUNT(*) as count FROM user', (err, result) => {
    if (err) throw err;
    res.render('home.ejs', { count: result[0].count });
  });
});

// 2. GET /user → Show all users
app.get('/user', (req, res) => {
  connection.query('SELECT id, username, email FROM user', (err, users) => {
    if (err) throw err;
    res.render('users.ejs', { users });
  });
});

// 3. PATCH /user/:id → Edit username
app.patch('/user/:id', (req, res) => {
  const { id } = req.params;
  const { username } = req.body;
  connection.query('UPDATE user SET username = ? WHERE id = ?', [username, id], (err) => {
    if (err) throw err;
    res.redirect('/user');
  });
});

// 4. DELETE /user/:id → Delete user
app.delete('/user/:id', (req, res) => {
  connection.query('DELETE FROM user WHERE id = ?', [req.params.id], (err) => {
    if (err) throw err;
    res.redirect('/user');
  });
});

app.listen(8080, () => console.log("Server listening on port 8080"));
```